

Fundamentos para el desarrollo de Agentes de IA

MSc. Jesus Gastañaduy

Agenda



1. ¿Qué es un agente de IA?

La base: un LLM conectado a datos y herramientas para resolver tareas



2. Componentes básicos de un agente

Modelo, System Prompt y Tools

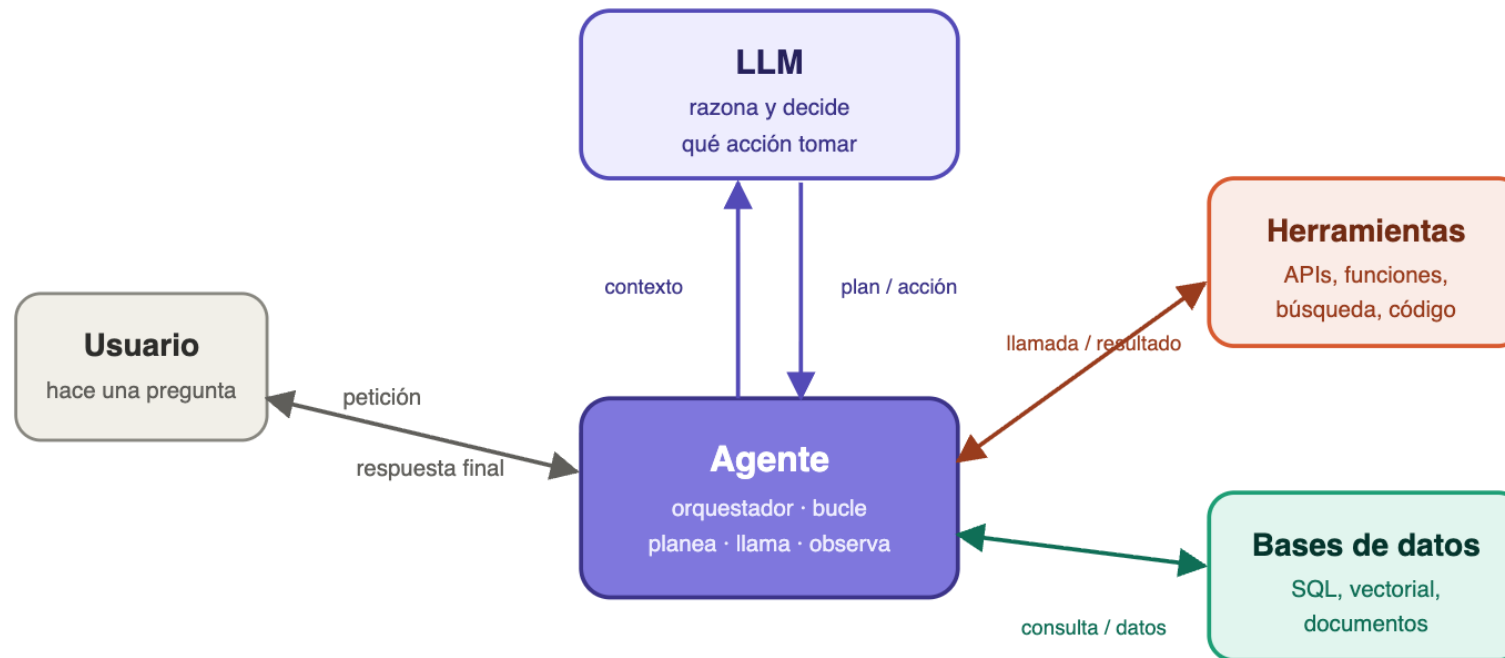


3. Introducción a Framework DeepAgents de LangChain

Un "harness" listo para construir agentes confiables

¿Qué es un agente de IA?

Pensemos en un agente de IA como un sistema que conecta un LLM con 1) bases de datos y 2) herramientas para resolver una tarea de manera agéntica.



El agente repite el ciclo **razonar** → **actuar** → **observar** hasta resolver la tarea

Componentes básicos para construir un agente



Modelo



System
Prompt



Tools



Componente 1

El Modelo

El LLM que impulsa el razonamiento, la planificación y la toma de decisiones del agente en cada paso del ciclo tool-calling.

- **Capacidad de tool-calling:** debe poder invocar funciones de forma confiable
- **Ventana de contexto:** cuánta información puede mantener en memoria de trabajo
- **Proveedor:** modelos propietarios (OpenAI GPT, Google Gemini, Anthropic Claude) u open source (DeepSeek, Google Gemma, Z.ai)

En DeepAgents

```
model="anthropic:claude-sonnet-4-6"
```

Un string simple, o una instancia de chat model de LangChain



Componente 2

El System Prompt

Define el rol, el objetivo y los límites del agente: es la instrucción persistente que guía cada decisión.

- Se carga al inicio de la conversación y se mantiene constante
- Buenas prácticas: instrucciones claras, contexto del dominio, restricciones explícitas
- En agentes complejos conviene versionarlo y probarlo

Ejemplo

```
system_prompt = (  
    "Eres un asistente de soporte. "  
    "Responde solo sobre nuestro "  
    "producto. Si no sabes la "  
    "respuesta, dilo con claridad."  
)
```

Rol, alcance y límites definidos en una sola instrucción



Componente 3

Tools

Operaciones o funciones básicas que se brindan como contexto al agente para que resuelva tareas específicas

Se agregan con `tools=[...]` al crear el agente

`ls``read_file``write_file``edit_file``glob``grep``execute`

Herramientas de filesystem incluidas por defecto en el harness de DeepAgents



Soporte para MCP (Model Context Protocol)

El agente puede conectarse a bases de datos, APIs y otros sistemas a través de servidores MCP, usando la misma interfaz de tools.

Introducción a Framework DeepAgents de LangChain



Modelo



System
Prompt



Tools

¿Por qué LangChain?

Un framework de building blocks para trabajar con LLMs de forma consistente



Interfaz estándar

El mismo código funciona con distintos proveedores de modelos: OpenAI, Anthropic, Google, etc.



Building blocks reutilizables

Modelos, prompts, herramientas, memoria y cadenas que se combinan entre sí



Ecosistema maduro

Cientos de integraciones: bases de datos, APIs, servidores MCP y más

¿Qué es el Framework DeepAgents?

deepagents es una librería lista para usar que toma las piezas de LangChain y las ejecuta sobre LangGraph, de modo que tus agentes ya vienen con las capacidades que necesitan para funcionar en producción, sin que tú tengas que armarlas desde cero.



Planificación de tareas integrada

El agente organiza su trabajo con una lista de tareas estructurada (write_todos)



Gestión de contexto

Lee, escribe y busca en archivos para manejar contexto extenso (AGENTS.md)



Generación de subagentes

Delega subtareas a agentes con contexto propio y aislado



Control humano

Human-in-the-loop: pausas para aprobación en pasos críticos (interrupt_on)

Un Deep Agent en pocas líneas

`create_deep_agent()` ensambla modelo, system prompt y tools con el harness completo

```
from deepagents import create_deep_agent

def get_weather(city: str) -> str:
    """Get weather for a given city."""
    return f"It's always sunny in {city}!"

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",    # 1. Modelo
    tools=[get_weather],                    # 3. Tools
    system_prompt="You are a helpful assistant", # 2. System Prompt
)

agent.invoke({
    "messages": [{"role": "user", "content": "what is the weather in sf"}]
})
```

DeepAgents no reemplaza a LangChain: se construye sobre él



FRAMEWORK

LangChain

Building blocks: modelos, prompts, herramientas y agentes personalizados (create_agent)



RUNTIME

LangGraph

Orquesta grafos con estado: ejecución durable, streaming, human-in-the-loop



HARNESS

DeepAgents

Librería lista para usar sobre LangGraph: planificación, subagentes, filesystem virtual

Setup



VS Code

Editor de código que usaremos durante todo el curso



GitHub Copilot

Asistente de código con IA integrado en VS Code



Gemini API Key

Credencial para llamar a los modelos de Google desde el código



Environment variables

Configuración local del proyecto, guardada en un archivo .env



UV Package Manager

Gestor de paquetes y entornos virtuales de Python

Fundamentos para el desarrollo de Agentes de IA

MSc. Jesus Gastañaduy